Open in app ↗

Medium Search Play Games Write 3

Welcome Offer Access to everything. Now 30% off. Upgrade now

Databricks + Data Engineering + Spark + Big Data + Python Programming +

Databricks Level 6 (Performance & Cost Optimization) — Making Databricks Pipelines Faster, Cheaper, and Defensible at Production Scale



Anurag Sharma 20 min read · 5 days ago

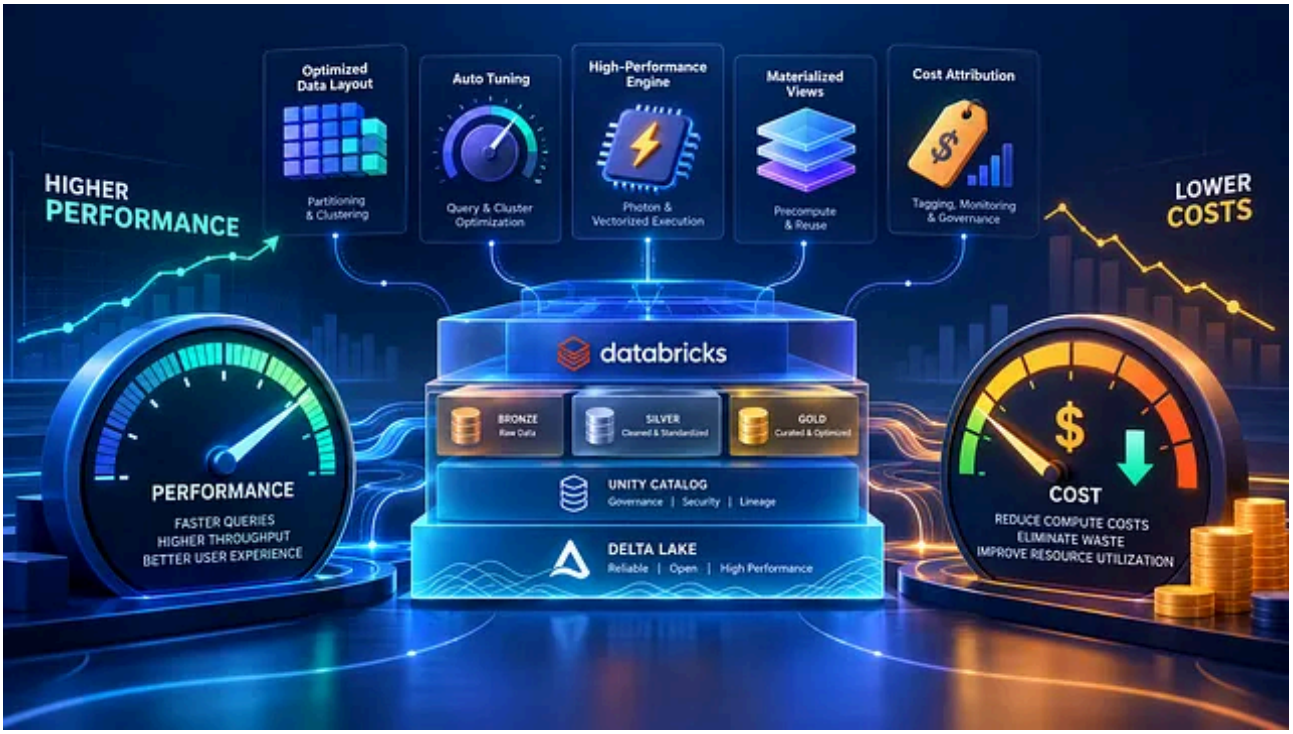
5 1



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform



Databricks Level 6 (Performance & Cost Optimization) is where “it works and we can see it working” becomes “it works *efficiently*, and we can prove what it costs.” Databricks Level 5 (Production Readiness & Observability) gave you visibility. Level 6 uses that visibility to cut waste without hurting reliability.

This blog covers:

- The real structure of a Databricks bill (DBUs plus cloud infrastructure)
- Workload characterization
- Compute sizing
- Adaptive Query Execution (AQE)
- Photon
- Data layout: partitioning, Liquid Clustering, file sizing and data skipping
- Query patterns
- Materialized views and caching



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

- Automatic maintenance: Predictive Optimization vs Auto Optimize
- Cost attribution through tags and system tables
- Benchmarking discipline

The goal is not a bag of tricks. It's a repeatable process: measure, find the bottleneck, change one thing, validate, document.

Who Should Read This Blog

1. **Data engineers whose pipelines work but whose bills keep climbing**
2. **Platform teams responsible for compute policies and cost governance**
3. **Anyone preparing for senior or staff-level data engineering interviews**
4. **Tech leads who must explain cloud spend to finance**
5. **Engineers who want Delta tables and Spark jobs to be genuinely fast**
6. **Teams moving from "it runs" to "it runs well"**
7. **Anyone who has tuned a Spark config and couldn't prove it helped**



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

The Bill That Arrives After the Architecture

Most cost conversations start late. They go like this:

- "Why did our Databricks bill double in two quarters?"
- "Why does this dashboard take 40 seconds?"
- "Why is the nightly job now finishing at 9 AM instead of 5 AM?"

Nobody chose to waste money. It accumulated through small decisions:

- A cluster nobody terminated
- A table that never got compacted
- A join that stopped fitting in memory when the data doubled
- An all-purpose cluster running a scheduled job because “it was already there”

Level 6 (Performance & Cost Optimization) is the discipline of making those decisions deliberately.

The Mindset: Performance and Cost Are the Same Problem, Mostly

On a usage-based platform, time is money. A query that runs half as long on the same cluster costs roughly half as much. That’s why the best optimizations improve both.

There are exceptions you need to know about:

Situation	Why speed and cost diverge
Photon	Photon workloads use a higher DBU rate, so it only saves money if the speedup beats the rate multiplier
Bigger cluster	Doubling workers rarely halves runtime, so the run gets faster but costs more
Spot instances	Cheaper per hour, but interruptions can lengthen runs
Caching and MVs	Save compute but add storage and refresh cost



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

So the real objective is **cost per unit of useful work at an acceptable latency**, not "make it fast" or "make it cheap" in isolation.

Step 1: Know What You Are Actually Paying For

You can't optimize a bill you can't read.

A Databricks bill has two parts

1. **Databricks charges:** DBUs consumed multiplied by a per-DBU price. The rate depends on the SKU (Jobs Compute, All-Purpose Compute, SQL, serverless and so on).
2. **Cloud infrastructure charges** (classic compute): the VMs, disks and networking billed by AWS, Azure or GCP.

Serverless products bundle the infrastructure cost into the DBU price. Classic compute doesn't.

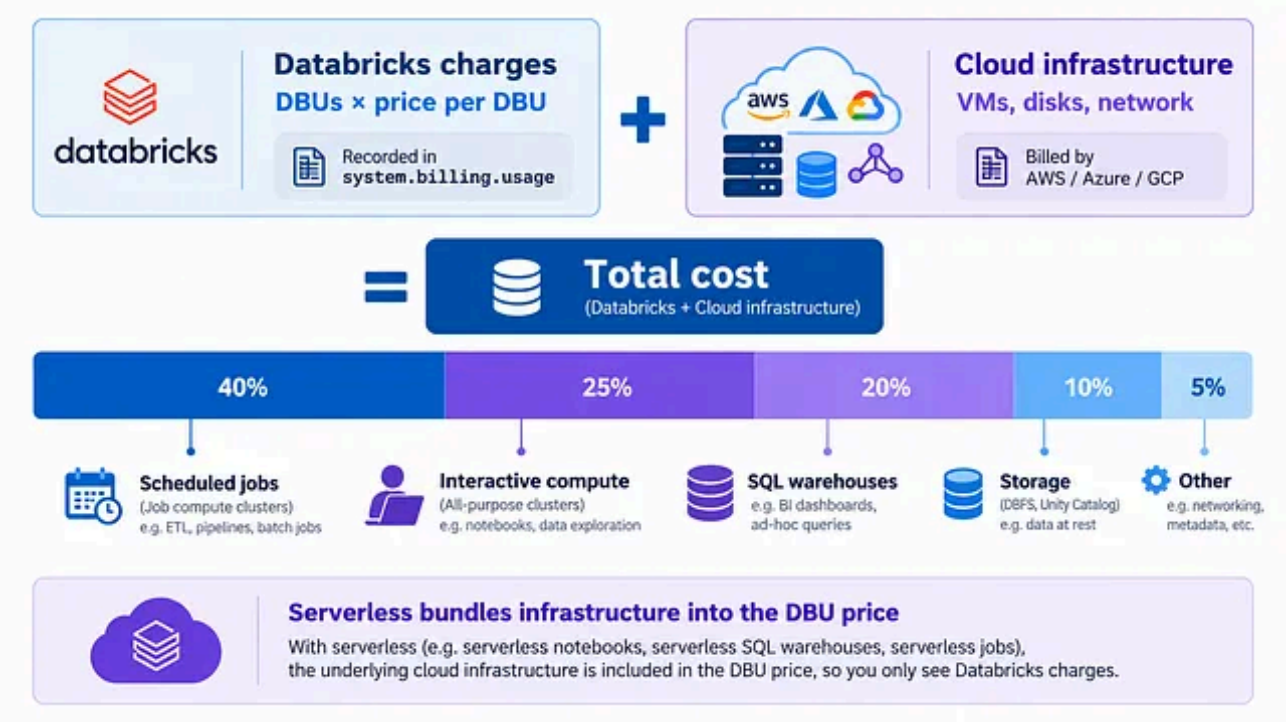
The billable usage system table, `system.billing.usage`, records **DBU usage only**. It doesn't include your cloud provider's VM charges. If you only look at Databricks usage on classic compute, you're seeing part of the real cost. Pull the cloud bill too, joined by cluster tags.



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform



Where the money usually goes

An illustrative breakdown (yours will differ):

```
Total monthly platform spend: $50,000

Compute (DBUs + VMs):         ~80%
  - Scheduled job compute       ~45%
  - Interactive / all-purpose   ~20%
  - SQL warehouses              ~15%
Storage (cloud object store):   ~12%
Everything else (network, etc.): ~8%
```



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

The point is the shape. Compute dominates, and interactive compute is where waste hides, because nobody owns it.

Why the compute type matters so much

The same code can cost very different amounts depending on where it runs.

Compute type	Best for	Cost watch-out
Jobs compute (job cluster)	Scheduled pipelines	Lower DBU rate than all-purpose; terminates when the run ends
All-purpose compute	Interactive development	Higher DBU rate; easy to leave running
SQL warehouse (serverless/pro/classic)	BI and SQL workloads	Size and auto-stop settings drive cost
Serverless compute	Spiky or hands-off	Pay for use, no idle VM management

The first quick win in many accounts: scheduled jobs running on all-purpose clusters. Move them to job clusters or serverless jobs and the DBU rate drops without touching a line of code.

Step 2: Measure Cost Correctly with System Tables

This is the corrected version of the cost query. It uses `system.billing.usage` together with `system.billing.list_prices`.

The core pattern: usage × price

```
-- Estimated list-price cost per team over the last 30 days
SELECT
    u.custom_tags['team']                          AS team,
    u.sku_name,
    SUM(u.usage_quantity)                          AS dbus,
    SUM(u.usage_quantity * lp.pricing.default)     AS est_list_cost_usd
FROM system.billing.usage u
JOIN system.billing.list_prices lp
  ON  u.sku_name    = lp.sku_name
  AND u.cloud       = lp.cloud
  AND u.usage_unit  = lp.usage_unit
  AND u.usage_end_time >= lp.price_start_time
  AND (lp.price_end_time IS NULL OR u.usage_end_time < lp.price_end_time)
WHERE u.usage_date >= current_date() - 30
GROUP BY u.custom_tags['team'], u.sku_name
ORDER BY est_list_cost_usd DESC;
```



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

Notes on this query:

- `custom_tags` is a map on the usage table. You read tags with `custom_tags['team']`, not by joining a tags table.
- The price join is time-bounded, because list prices change over time.
- It gives list price. If you have negotiated discounts, actual cost will be lower.
- Usage corrections arrive as retraction and restatement records. Summing handles them correctly because retractions carry negative quantities. The docs describe `record_type` and note that ORIGINAL records are available within about 12 hours.

Cost per job

`usage_metadata` carries job and cluster identifiers. Use them to rank pipelines by spend:

```
SELECT
    u.usage_metadata.job_id                        AS job_id,
    COUNT(DISTINCT u.usage_metadata.job_run_id)  AS runs,
    SUM(u.usage_quantity)                          AS dbus,
    SUM(u.usage_quantity * lp.pricing.default)   AS est_list_cost_usd,
    SUM(u.usage_quantity * lp.pricing.default)
      / NULLIF(COUNT(DISTINCT u.usage_metadata.job_run_id), 0) AS cost_per_run
FROM system.billing.usage u
JOIN system.billing.list_prices lp
  ON  u.sku_name    = lp.sku_name
  AND u.cloud       = lp.cloud
  AND u.usage_unit = lp.usage_unit
  AND u.usage_end_time >= lp.price_start_time
  AND (lp.price_end_time IS NULL OR u.usage_end_time < lp.price_end_time)
WHERE u.usage_date >= current_date() - 30
  AND u.usage_metadata.job_id IS NOT NULL
GROUP BY u.usage_metadata.job_id
ORDER BY est_list_cost_usd DESC
LIMIT 20;
```



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

Treat the 20 rows this returns as your optimization backlog. Databricks publishes a set of ready-made cost queries on this pattern in its “Top 5 System Table Queries” post.

Fixing the duration math (the `* 3600` error)

The old draft multiplied seconds by 3600, which converts in the wrong direction. Seconds divide by 3600 to get hours. For your own `monitoring.costs` table from Blog 5:

```
SELECT
    pipeline_name,
    COUNT(*)                                    AS run_count,
    SUM(duration_seconds) / 3600.0              AS total_run_hours,
    SUM(duration_seconds * node_count) / 3600.0 AS total_node_hours,
    SUM(cost_usd)                               AS total_cost,
    AVG(duration_seconds)                       AS avg_duration_seconds
FROM monitoring.costs
WHERE timestamp >= current_date() - 30
GROUP BY pipeline_name
ORDER BY total_cost DESC
LIMIT 10;
```



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

- `total_run_hours` is wall-clock hours.
- `total_node_hours` multiplies each run's duration by its node count (workers plus driver), so you capture how big the cluster was, not just how long it ran.

Tag everything, or you can’t attribute anything

```
custom_tags:
  team: "data-engineering"
  cost_center: "analytics"
```

```
    environment: "production"
    project: "revenue_platform"
```

Enforce tags with cluster policies so they can't be skipped. Untagged spend can't be charged back to anyone.

Step 3: Workload Characterization — Know Before You Tune

Different workloads need different optimizations. Classify before you touch anything.

Workload	Profile	Primary optimization levers
Batch ETL	Scheduled, large scans, heavy shuffles	Data layout, join strategy, cluster sizing, spot
Streaming / incremental	Continuous or triggered micro-batches	Trigger interval, state size, file compaction
Interactive / ad hoc	Unpredictable, many users	Auto-termination, serverless, query limits
BI / dashboards	Repetitive, latency-sensitive	Materialized views, clustering, warehouse sizi
ML / feature engineering	CPU- or memory-heavy	Node type choice, GPU only when needed
Backfills	Rare, huge, time-bounded	Temporary larger clusters, chunked runs



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

Build a profile for each top workload

```
query_profile = {
    "name": "daily_revenue_aggregation",
    "type": "batch_etl",
    "frequency": "daily",
    "typical_runtime_minutes": 30,
    "input_gb": 50,
    "scan_pattern": "date_filtered_scan",
    "shuffle_intensity": "high",
    "spill_to_disk": False,
    "avg_cpu_utilization_pct": 45,
```

```
        "cost_per_run_usd": 12.50,
    }
```

This profile answers one question: is the bottleneck I/O, CPU, memory, shuffle, or skew? Each has a different fix:

Symptom	Likely bottleneck	First thing to try
Large "bytes read" for a small result	I/O / bad layout	Clustering, partition pruning, column pruning
Low CPU, long runtime	I/O-bound or waiting on shuffle	Layout, fewer shuffles, faster storage
High CPU, long runtime	Compute-bound	Photon, more cores, better algorithm
Spill to disk	Memory pressure	Memory-optimized nodes, fewer partitions per task, fix skew
One task much slower than the rest	Data skew	AQE skew join, salting, better key
Many tiny tasks	Small files	OPTIMIZE, auto compaction

The 80/20 rule

A small fraction of workloads usually drives most of the spend. Use the per-job cost query from Step 2 and start there. Optimizing your 50th most expensive job is nearly always a waste of a week.



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

Step 4: Compute Sizing — What You Actually Control on Databricks

This section corrects a common misunderstanding.

On open-source Spark you tune `spark.executor.cores` and `spark.executor.memory` directly. On Databricks, each worker node runs one executor sized to the node, and you can't change those values at runtime with `spark.conf.set`. Executor size follows from the node type you choose.

So your sizing levers are:

1. Node type (CPU, memory, local SSD, GPU)
2. Number of workers (fixed or autoscaling)
3. Runtime engine (Photon or standard)
4. Availability (on-demand or spot)
5. Serverless vs classic

Choosing a node family

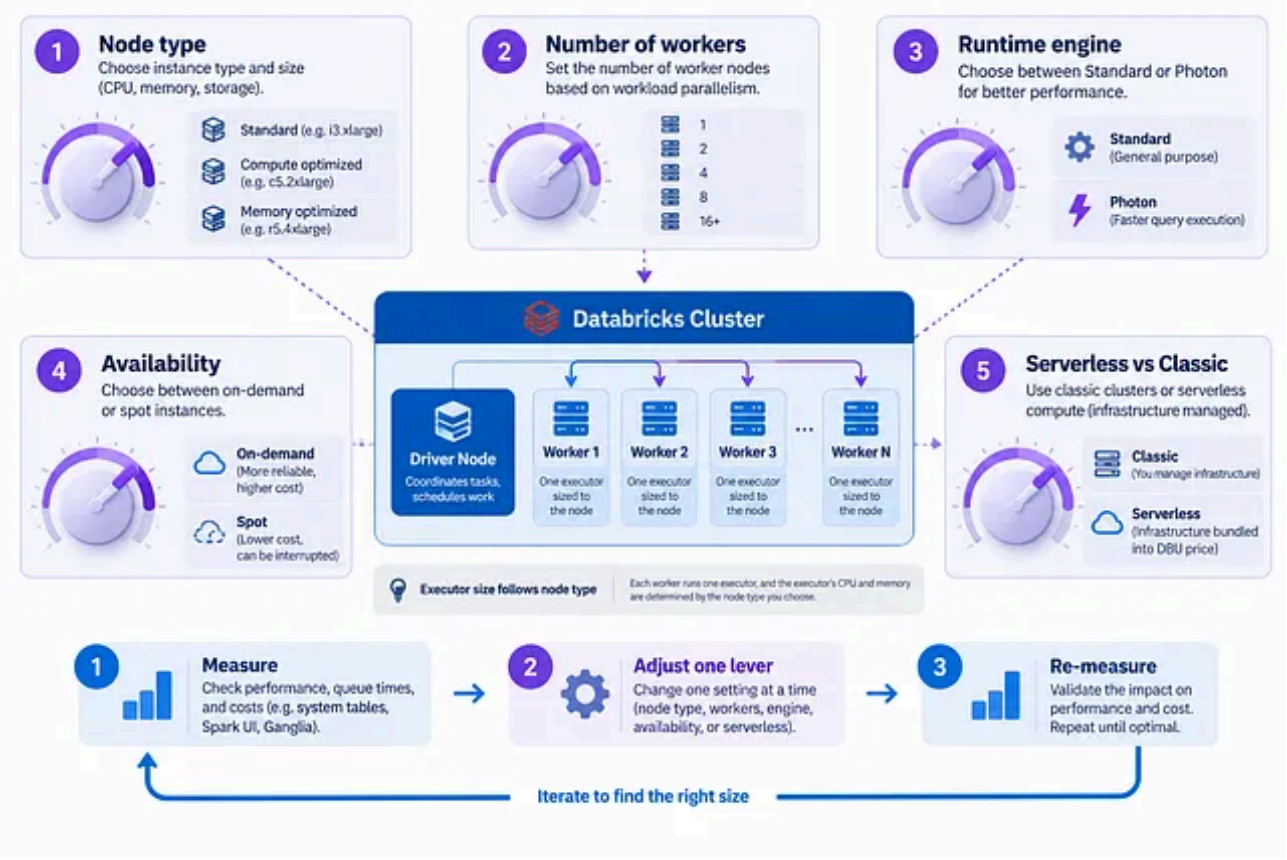
Family	Good for	Signals you need it
General purpose	Mixed ETL	Balanced CPU and memory use
Memory-optimized	Big joins, wide aggregations, caching	Spill to disk, GC pauses, OOM
Compute-optimized	CPU-bound transforms, Photon-heavy SQL	High CPU, little spill
Storage-optimized (local NVMe)	Repeated reads of the same data	Benefits from the disk cache
GPU	Deep learning only	Not for ordinary ETL



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform



A sizing workflow that works

1. Run the job once on a reasonable default.
2. Open the cluster metrics and the Spark UI. Check CPU, memory, spill an shuffle.
3. Low CPU and no spill: you're over-provisioned or I/O-bound. Try fewer c smaller nodes.
4. Spill or OOM: move to memory-optimized nodes, or fix skew first.
5. High CPU and a missed SLA: add workers or enable Photon.
6. Re-run, compare against the baseline, and keep only what helped.

Utilization from system tables

If the compute system tables are enabled in your account, you can check node utilization without leaving SQL. Column names can change, so confirm the schema in your workspace before relying on this:



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

```
-- Verify columns in your workspace first: DESCRIBE system.compute.node_timeline
SELECT
    cluster_id,
    AVG(cpu_user_percent + cpu_system_percent) AS avg_cpu_pct,
    AVG(mem_used_percent)                      AS avg_mem_pct
FROM system.compute.node_timeline
WHERE start_time >= current_date() - 7
GROUP BY cluster_id
ORDER BY avg_cpu_pct ASC
LIMIT 20;
```

Rule of thumb: a batch cluster averaging under 40–50% CPU with no spill is probably oversized.

Autoscaling: useful, not magic

```
new_cluster:
  spark_version: "15.4.x-scala2.12"
  node_type_id: "i3.xlarge"
  autoscale:
    min_workers: 2
    max_workers: 8
```



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

Autoscaling helps with variable load. For short, predictable batch jobs a fixed-size cluster is often cheaper and easier to reason about, because scale up has startup lag and the extra nodes may arrive after the heavy stage has passed. Profile before assuming autoscaling saves money.

Auto-termination (corrected)

The earlier example put a `singleNode` profile under an autoscale block and called it auto-termination. That was wrong. Here is the correct picture.

All-purpose (interactive) clusters use `autotermination_minutes`:

```
{
  "cluster_name": "analyst_adhoc",
  "spark_version": "15.4.x-scala2.12",
  "node_type_id": "i3.xlarge",
  "autoscale": { "min_workers": 1, "max_workers": 4 },
  "autotermination_minutes": 30
}
```

Job clusters need no auto-termination setting. A job cluster is created for the run and terminates when the run finishes:

```
job_clusters:
  - job_cluster_key: etl_cluster
    new_cluster:
      spark_version: "15.4.x-scala2.12"
      node_type_id: "i3.xlarge"
      num_workers: 4
tasks:
  - task_key: silver_orders
    job_cluster_key: etl_cluster
    notebook_task:
      notebook_path: "/Repos/analytics/silver_orders"
```



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

SQL warehouses use an **auto stop** setting (minutes of inactivity).

Set short idle timeouts for interactive compute. A 10–30 minute auto-termination on an all-purpose cluster is common, and it prevents the classic “forgotten cluster over the weekend” bill.

Spot instances (with a safe pattern)

```
new_cluster:
  aws_attributes:
```

```
        first_on_demand: 1
        availability: "SPOT_WITH_FALLBACK"
```

- `first_on_demand: 1` keeps the driver on an on-demand instance, so an interruption doesn't kill the whole job.
- `SPOT_WITH_FALLBACK` falls back to on-demand if spot capacity isn't available.
- This is a good fit for idempotent batch ETL (Blog 4). Avoid it for latency-critical or non-restartable work.

Serverless: when it wins

Serverless removes cluster sizing and idle-VM management. It tends to win for spiky, short or interactive workloads where startup time and idle waste dominate. For steady, heavy, well-tuned batch jobs, classic job clusters with spot can still be cheaper. Compare actual cost per run rather than assuming.



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

Step 5: Adaptive Query Execution — Free Performance You Should Understand

AQE re-optimizes a query plan at runtime using real statistics from completed stages. It is **enabled by default in recent Databricks Runtime versions**, but you should know what it does and how to verify it.

```
print(spark.conf.get("spark.sql.adaptive.enabled"))
```

What AQE does

1. Coalesces shuffle partitions

Shuffle partition count is a guess made before data is seen. AQE merges small post-shuffle partitions at runtime.

```
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
```

This is why setting `spark.sql.shuffle.partitions` slightly high is usually safe with AQE. Recent Databricks runtimes also support `auto` for this setting, which lets the platform choose the partition count.

2. **Switches to broadcast joins at runtime**

If one side of a join turns out to be small after filtering, AQE can convert a sort-merge join to a broadcast join.

3. **Splits skewed partitions**

```
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

When one partition is much larger than the median, AQE splits it so multipl tasks share the work.



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

Detecting skew yourself

In the Spark UI stage view:

- Compare **max task duration to median task duration.** A large gap means skew.

- Look at **shuffle read size per task.** One huge outlier means a hot key.
- Look at **spill (memory and disk)** columns.

If AQE doesn't fix it (extreme skew on a single key), consider:

- Salting the hot key
- Pre-aggregating before the join
- Handling the hot key as a separate path (for example, a null or default key that dominates)

Don't quote precise AQE speedup percentages in interviews. Benefit depends entirely on the workload. Say "often material for skewed or over-partitioned shuffles, and I verify with the Spark UI."



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

Step 6: Photon — When It Pays for Itself

Photon is Databricks' native vectorized query engine. It accelerates SQL and DataFrame workloads, especially scans, aggregations, joins and Delta writes

```
new_cluster:
  spark_version: "15.4.x-scala2.12"
  node_type_id: "i3.xlarge"
  runtime_engine: "PHOTON"
  num_workers: 4
```

The cost trade-off most people miss

Photon-enabled compute is billed at a **higher DBU rate.** So:

> *Photon saves money only if the speedup is larger than the price premium.*

Scenario	Likely outcome
SQL-heavy ETL with big scans/joins/aggregations	Often a clear net win
Python UDF-heavy or tiny jobs	Little benefit; may cost more
Workloads dominated by I/O wait	Limited benefit

How to decide: run the same job with and without Photon, record runtime and cost per run, and compare. Don't assume.

Step 7: Data Layout — The Highest-Leverage Optimization

How data is laid out on storage usually matters more than any config. The principle is simple: **read less data**.



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

The three levers

1. **Partitioning** (directory-level pruning)
2. **Clustering** (file-level data co-location, via Liquid Clustering)
3. **File sizing and statistics** (data skipping)

Partitioning: use it sparingly

```
CREATE TABLE silver.orders_partitioned (
    order_id    STRING,
    customer_id STRING,
    amount      DECIMAL(12,2),
    order_date  DATE
```

```
)
PARTITIONED BY (order_date);
```

Databricks guidance for Delta tables is to avoid partitioning unless the table is large (on the order of a terabyte or more) and each partition holds a substantial amount of data (around a gigabyte or more). Over-partitioning creates many small files and slows everything down. Partitioning small tables by date is a very common mistake.

Good partition candidates: date columns on very large tables, queried with date filters. Bad partition candidates: high-cardinality columns (`order_id` , `customer_id`), continuous values, columns rarely filtered on.

Liquid Clustering: the modern default for new tables

Liquid Clustering replaces both partitioning and ZORDER for most new Delta tables. You declare clustering keys, and layout is maintained incrementally.



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

```
CREATE TABLE silver.orders_clustered (
    order_id    STRING,
    customer_id STRING,
    amount      DECIMAL(12,2),
    order_date  DATE,
    region      STRING
)
CLUSTER BY (order_date, customer_id);
```

Change keys later without rewriting the whole table:

```
ALTER TABLE silver.orders_clustered CLUSTER BY (customer_id, region);
```

Cluster on demand with `OPTIMIZE`:

```
OPTIMIZE silver.orders_clustered;
```

You can also let Databricks choose keys for Unity Catalog managed tables:

```
ALTER TABLE silver.orders_clustered CLUSTER BY AUTO;
```

Important constraints (this fixes the old comparison table):

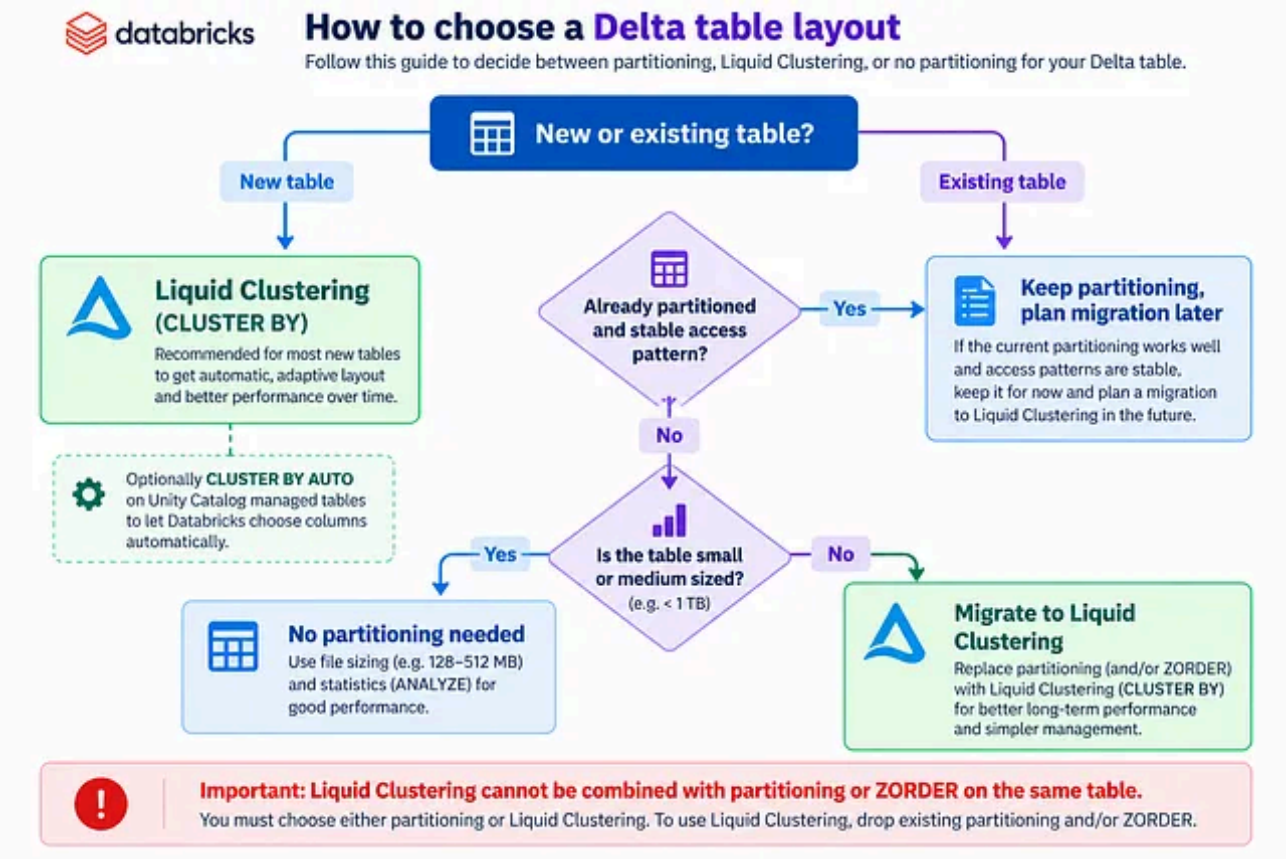
- Liquid Clustering **can't be combined with partitioning or ZORDER on the same table**. It replaces them.
- Keep the key list short. A few columns, chosen from your real filter and join patterns, is the right scale.
- Order of keys doesn't matter in the way it does for ZORDER. Pick columns your queries filter on.



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform



Which approach, when (corrected)

Scenario	Recommendation
New table, evolving query patterns	Liquid Clustering
Queries filter on high-cardinality columns (customer_id, product_id)	Liquid Clustering
Queries filter on date AND a high-cardinality column	Liquid Clustering with both columns as keys
Very large legacy table already partitioned, stable access pattern	Keep partitioning; migrate when practical
Existing table using ZORDER	Plan a move to Liquid Clustering
Small or medium table	Neither is needed; rely on file sizing and stats



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

ZORDER (legacy)

```
OPTIMIZE silver.orders ZORDER BY (customer_id, order_date);
```

Still works on existing tables, but it rewrites data on each run and isn't compatible with Liquid Clustering. Keep to a small number of columns. Many columns dilute the benefit.

File sizing and the small files problem

Too many tiny files mean high metadata overhead and slow scans. Inspect a table:

```
DESCRIBE DETAIL silver.orders;
```

This returns `numFiles` and `sizeInBytes`. Divide to get average file size. Fix with:

```
OPTIMIZE silver.orders;
```

`OPTIMIZE` compacts small files toward a larger target size (up to about 1 GB per file by default, with Databricks autotuning target size for smaller tables). Don't chase one magic number. Watch for tables with thousands of files averaging a few MB.

Data skipping and statistics

Delta records min/max statistics per file for the first columns of the table and uses them to skip files that can't match a filter.

```
-- Index specific columns that you actually filter on
ALTER TABLE silver.orders SET TBLPROPERTIES (
```



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

```
    'delta.dataSkippingStatsColumns' = 'order_date,customer_id,region'
);
```

By default, statistics cover the first 32 columns. Two practical consequences:

- Put frequently filtered columns early in the schema, or name them explicitly as above.
- Long string columns and wide tables can make statistics collection expensive. Exclude columns that you never filter on.

VACUUM and storage

```
VACUUM silver.orders;   -- default retention: 7 days
```

`VACUUM` deletes files no longer referenced by the table and older than the retention period. Trade-off: shorter retention saves storage, but limits time travel and can break long-running readers. Don't drop retention below a saf value without understanding concurrent readers and streaming jobs.



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

Step 8: Automatic Maintenance — Predictive Optimization vs Auto Optimize

The earlier draft treated these as one feature. They are two different features.

Predictive Optimization (table maintenance, automated)

A Unity Catalog feature that automatically runs maintenance on **managed tables:** `OPTIMIZE` (compaction and clustering), `VACUUM`, and statistics

collection (`ANALYZE`). It decides when and what to run based on table usage.



It is enabled at the account, catalog or schema level:

```
ALTER SCHEMA main.silver ENABLE PREDICTIVE OPTIMIZATION;
```

Why it matters for cost: you stop paying engineers (or scheduled jobs) to run `OPTIMIZE` and `VACUUM` on the right tables at the right frequency. It runs on Databricks-managed serverless compute. Check the current docs for pricing and supported table types before enabling it widely.



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

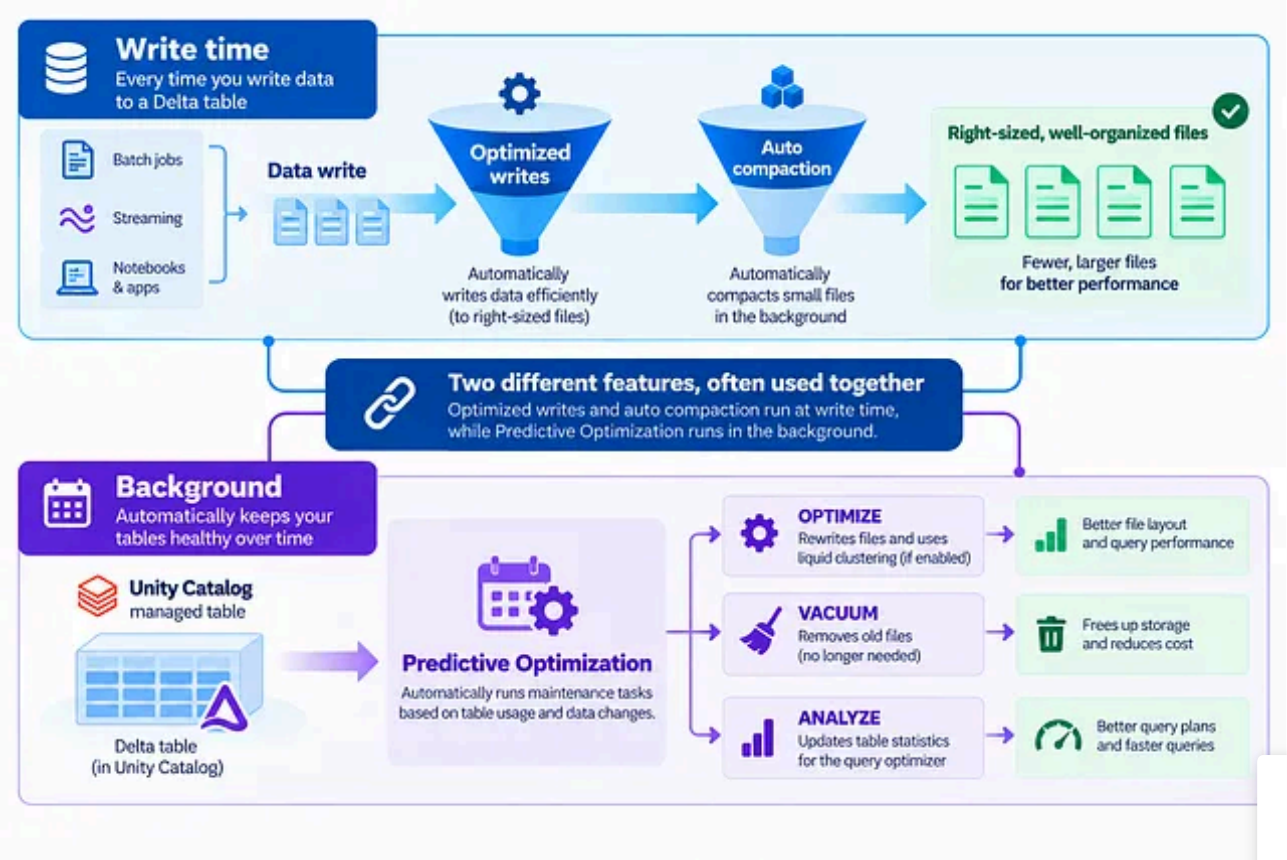
Auto Optimize (write-time file sizing)

Table properties that act **during writes**:

```
ALTER TABLE silver.orders SET TBLPROPERTIES (
    'delta.autoOptimize.optimizeWrite' = 'true',
    'delta.autoOptimize.autoCompact'   = 'true'
);
```

- **Optimized writes** shuffle data before writing to produce fewer, better-sized files.
- **Auto compaction** compacts small files after a write completes.

These help with the small-files problem from frequent small writes (streaming, micro-batch merges), but they add some write-time overhead.



How they fit together

Feature	When it acts	What it does	Scope
Optimized writes / auto compaction	During or right after a write	Better file sizes at write time	Table or session property
Predictive Optimization	Background, scheduled by the platform	OPTIMIZE, VACUUM, ANALYZE	UC managed tables, enabled by schema/catalog/account
Manual `OPTIMIZE` / `VACUUM`	When you schedule it	Same operations, under your control	Any Delta table

Practical guidance: for Unity Catalog managed tables, Predictive Optimization is the low-effort default. For external tables or special cases, schedule `OPTIMIZE` and `VACUUM` yourself.



Anura
149 foll
Data Er
Exp. Pa
pipeline
perform

Step 9: Query Optimization Patterns

Layout and sizing set the ceiling. Query design decides whether you reach it.

Pattern 1: Filter early and let pruning work

Write filters in a form the engine can push down.

```
-- Good: direct predicate on the column, enables file skipping
SELECT order_id, amount
FROM silver.orders
WHERE order_date >= '2026-10-01' AND order_date < '2026-11-01';

-- Risky: wrapping the column in a function can defeat pruning
SELECT order_id, amount
FROM silver.orders
WHERE date_format(order_date, 'yyyy-MM') = '2026-10';
```

The optimizer already pushes many filters down, so don't rewrite working queries just because a filter "looks late." Check the plan.

Pattern 2: Select only the columns you need

```
-- Avoid
SELECT * FROM silver.orders WHERE order_date = '2026-10-15';

-- Prefer
SELECT order_id, customer_id, amount
FROM silver.orders
WHERE order_date = '2026-10-15';
```

Columnar formats read only requested columns, so this directly reduces I/O.



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

Pattern 3: Join strategy

```
SELECT /*+ BROADCAST(c) */
    o.order_id,
    c.customer_name
FROM silver.orders o
JOIN silver.customers c
  ON o.customer_id = c.customer_id;
```

- Broadcasting a genuinely small dimension table avoids a shuffle of the large side.
- Don't broadcast something large. It consumes memory on every executor and can cause OOM.
- With AQE and good statistics, the engine often makes this choice itself. Hints are for cases where you know better.

Pattern 4: Reduce shuffles

Shuffles move data across the network and are often the dominant cost.

- Aggregate before joining when you can
- Avoid unnecessary `DISTINCT`, `ORDER BY` and `repartition` calls
- Avoid repartitioning to the same key the data is already organized by
- Don't `collect()` large results to the driver



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

Pattern 5: Avoid Python UDFs on hot paths

Row-by-row Python UDFs serialize data between the JVM and Python and bypass many optimizations. Prefer built-in functions, SQL expressions, or pandas/Arrow-based UDFs when you truly need Python.

Pattern 6: Incremental over full recompute

The cheapest data to process is data you've already processed.

- Use `MERGE` or `replaceWhere` for only the affected partitions
- Use Auto Loader to ingest only new files
- Use incremental materialized views instead of full rebuilds

Pattern 7: Read the plan

```
EXPLAIN FORMATTED
SELECT ...
```

Check:

- Is the filter pushed down into the scan?
- How many files and bytes are read?
- Which join type was chosen?
- Is Photon being used?

The **Query Profile** in Databricks SQL gives a visual breakdown. It shows tim per operator, rows, bytes read, files pruned and spill, and it is usually the fastest route to the real bottleneck.



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

Step 10: Caching and Materialized Views

Both avoid recomputation, but in different ways.

Caching, the right way on Databricks

- **Disk cache (automatic):** on supported node types, Databricks caches remote Parquet/Delta data on local SSD. You don't manage it. Storage-optimized and NVMe-equipped nodes benefit most.
- **`df.cache()` (Spark cache):** use rarely. It holds data in executor memory, can cause memory pressure, and isn't supported on serverless compute.

```
# Only when a DataFrame is reused multiple times in the same job
df = spark.table("silver.orders").filter("status = 'DELIVERED'").cache()
df.count()  # materializes
# ... multiple reuses ...
df.unpersist()
```

Don't cache: data used once, very large datasets, or anything you can cheaply re-read from a well-laid-out Delta table.

Materialized views

For repeated aggregations, a materialized view stores the result and refreshes it, often incrementally, instead of recomputing from scratch.



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

```
CREATE MATERIALIZED VIEW gold.daily_revenue_mv
AS
SELECT
    order_date,
    country,
    SUM(amount) AS total_revenue,
    COUNT(*)    AS order_count
FROM silver.orders
GROUP BY order_date, country;
```

Refresh it on demand or on a schedule:

```
REFRESH MATERIALIZED VIEW gold.daily_revenue_mv;
```

Things to verify in current docs for your environment: materialized views are a Unity Catalog feature, run their refreshes on serverless, and whether a given query is refreshed incrementally or fully depends on the query shape and the source tables. Don't assume incremental refresh. Check the refresh history.

Use a materialized view when:

- The same aggregation is queried repeatedly (dashboards)
- The source changes incrementally
- Recompute cost is higher than storage plus refresh cost

Avoid it when:

- Queries are one-off or highly variable
- The source is fully rewritten on every run
- Freshness requirements are tighter than a refresh cycle can meet



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

Decision table

Need	Tool
Faster repeated reads of raw data	Disk cache (automatic)
Reuse within one job	`cache()` sparingly, or write an intermediate Delta table
Fast, repeated dashboard aggregations	Materialized view or a Gold table
Full control over refresh logic	Gold table built by your pipeline

Step 11: Cost Attribution and Chargeback

Optimization without accountability doesn't stick. Teams that don't see their costs don't reduce them.

Tiers of maturity

Tier	What exists	Behavior it creates
1. Blind	One bill, no breakdown	Nobody owns cost
2. Tagged	Tags on compute and jobs	Teams can see their share
3. Showback	Regular cost reports per team	Teams start asking questions
4. Chargeback	Costs allocated to team budgets	Teams optimize actively

Most organizations should aim for Tier 3 first. Chargeback adds friction and disputes, so adopt it when showback alone isn't changing behavior.

How to get there

1. Define a tag standard: `team`, `cost_center`, `environment`, `project`.
2. Enforce it with **cluster policies** (required tags, allowed node types, max workers, mandatory auto-termination).
3. Use budget policies for serverless usage where available.
4. Build a monthly report from `system.billing.usage` (Step 2).
5. Add the cloud provider's VM costs, joined by tags, for classic compute.
6. Publish it on a shared dashboard and review it regularly.

A monthly allocation table



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

```
chargeback_record = {
    "team": "data-engineering",
    "month": "2026-10",
    "databricks_dbu_cost_usd": 9500,
    "cloud_infra_cost_usd": 4000,
    "storage_cost_usd": 800,
    "total_cost_usd": 14300,
    "job_runs": 250,
    "cost_per_run_usd": 57.2,
}
```

Unit economics like **cost per run, cost per TB processed and cost per dashboard refresh** are far more useful than raw totals, because they show efficiency independent of growth.

Guardrails with cluster policies

Policies stop waste before it starts: cap `num_workers`, require `autotermination_minutes` on all-purpose clusters, restrict to approved node types, and require tags. This is the cheapest cost control available, because i prevents problems instead of reporting on them.



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

Step 12: Benchmarking — Prove It Worked

Optimization without measurement is guesswork. Here is the loop:

```
1. Baseline  → 2. Find bottleneck → 3. Change ONE thing
      ↑                                        ↓
6. Document  ← 5. Keep or revert ←  4. Re-measure
```

Rules for honest benchmarks

1. **Change one variable at a time.** Otherwise you can't attribute the gain.

2. **Run more than once.** Cloud performance varies. Compare medians, not single runs.
3. **Separate cold and warm runs.** The disk cache makes second runs faster for reasons unrelated to your change.
4. **Use realistic data volume.** Optimizations on a 1 GB sample may not transfer to 1 TB.
5. **Record cost, not just time.** Use cost per run alongside duration.

A simple benchmark record

```
from datetime import datetime

benchmark = {
    "pipeline": "silver_orders_daily",
    "change": "enable Photon",
    "timestamp": datetime.now(),
    "baseline_median_seconds": 1800,
    "new_median_seconds": 1050,
    "baseline_cost_usd": 12.50,
    "new_cost_usd": 10.10,
    "runs_per_variant": 5,
}

time_gain_pct = (1 - benchmark["new_median_seconds"] / benchmark["baseline_med
cost_gain_pct = (1 - benchmark["new_cost_usd"] / benchmark["baseline_cost_usd"
print(f"Time: -{time_gain_pct:.0f}%  Cost: -{cost_gain_pct:.0f}%")
```



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

A pragmatic threshold

If a change gives under about 10% improvement in time or cost, ask whether the added complexity is worth it. This is a judgment call, not a law. A 5% gain on your most expensive job can still be worth thousands of dollars a year.

Guard against regressions

Record baselines in your Level 5 monitoring tables and alert when a pipeline's duration or cost per run drifts materially from its baseline. Optimizations decay as data grows and code changes.

Step 13: Optimization Anti-Patterns

Anti-pattern	Why it hurts	Better approach
Scheduled jobs on all-purpose clusters	Higher DBU rate, idle risk	Job clusters or serverless jobs
No auto-termination on interactive clusters	Paying for idle compute	`autotermination_minutes`, enforced by policy
Partitioning small tables	Tiny files, slow queries	No partitioning, or Liquid Clustering
Partitioning by a high-cardinality column	Millions of directories	Cluster by that column instead
Combining partitioning with Liquid Clustering	Not supported on the same table	Pick one
Blindly setting `spark.executor.*` on Databricks	Ignored or unsupported at runtime	Choose node type and count
Caching everything	Memory pressure, slower jobs	Cache only reused data
Broadcasting large tables	Executor OOM	Broadcast only small dimensions
Never running OPTIMIZE or VACUUM	Small files and storage bloat	Predictive Optimization or a schedule
Treating Photon as always cheaper	Higher DBU rate can negate the speedup	Benchmark cost per run
Tuning without a baseline	Can't prove anything helped	Measure first, change one thing
Optimizing low-cost jobs	Effort with no impact	Start with the top-cost list
Untagged compute	No attribution	Tag policy enforced by cluster policies



Anura
149 foll
Data Er
Exp. Pa
pipeline
perform

Step 14: A Worked Example — Cutting a Nightly Job's Cost

A realistic story shows how the pieces combine.

Situation: the nightly `silver_orders` job runs for 95 minutes on an all-purpose cluster with 12 workers. Cost per run is high, and it sometimes

misses its 6 AM SLA.

Step 1: Measure. The per-job cost query puts it in the top 3 spenders. Cluster metrics show about 35% average CPU, with no spill.

Step 2: Read the plan. Query Profile shows most time in scanning `bronze.orders`, reading about 800 GB to produce a result from a single day of data.

Step 3: Fix the biggest lever first (layout).

- Bronze was unpartitioned and unclustered, with many small files.
- Switched the table to `CLUSTER BY (order_date)` and ran `OPTIMIZE`.
- Bytes read per run dropped sharply, and runtime fell to roughly 40 minutes in testing.

Step 4: Right-size compute.

- With the scan cheaper and CPU low, 12 workers was oversized.
- Tried 6 workers, re-benchmarked, and kept it because the SLA held.

Step 5: Move compute type.

- Moved from an all-purpose cluster to a job cluster with `SPOT_WITH_FALLBACK` and an on-demand driver.

Step 6: Validate and record. Five runs per variant, median compared. The new baseline went into the monitoring tables with an alert on drift.



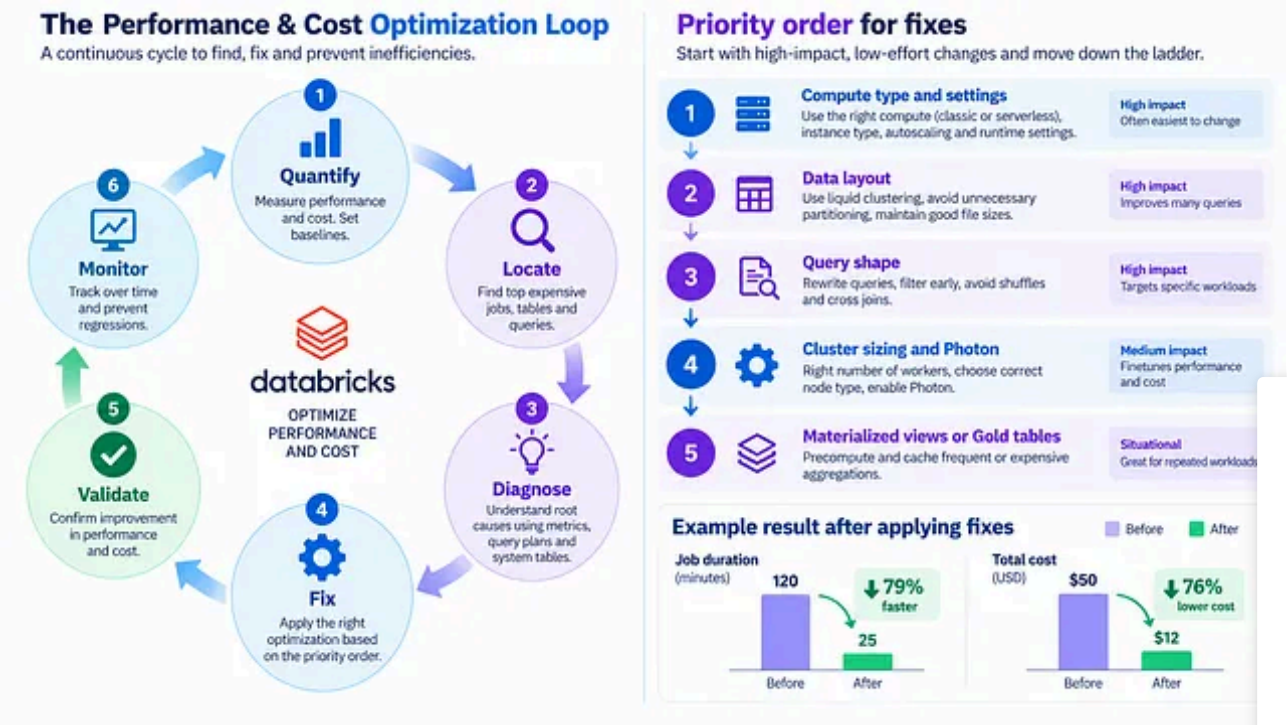
Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

Result (illustrative): runtime, cluster size and DBU rate all dropped, so the savings multiply. The layout fix was the largest single contributor. Notice the order: **layout, then sizing, then compute type, then fine-tuning**. Not the reverse.

Step 15: The Optimization Playbook



Use this whenever someone says “it’s slow” or “it’s expensive.”

1. **Quantify.** What is the runtime and cost per run? What is the target?
2. **Locate.** Is it in the top-cost list? If not, deprioritize.
3. **Diagnose.** Spark UI or Query Profile: I/O, CPU, memory, shuffle, or skew?
4. **Fix in priority order:**



— Compute type and settings (job vs all-purpose, auto-termination, serverless)

— Data layout (clustering, file sizes, statistics)

— Query shape (columns, joins, shuffles, UDFs, incremental logic)

— Cluster sizing and Photon

— Materialized views or Gold tables for repeated reads

5. **Validate.** Re-run, compare medians, check both time and cost.

6. **Document.** What changed, why, and the new baseline.

7. **Monitor.** Alert on regression.



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

Step 16: Building a Cost-Aware Culture

Tools don't sustain optimization. Habits do.

Cadence	Activity
Weekly	Review the top 10 jobs by cost and any anomalies
Monthly	Showback report to each team; review idle and untagged spend
Quarterly	Review layout and clustering on the largest tables; revisit serverless vs classic; revisit policies
Per release	Include expected cost impact in design reviews for new pipelines

Useful KPIs:

- Cost per pipeline run (trend)
- Cost per TB processed
- Percentage of spend that is tagged
- Percentage of interactive compute with auto-termination enforced
- Share of scheduled work on job or serverless compute
- Top-10 job cost trend

Step 17: Interview Questions at Level 6 (Performance & Cost Optimization)

Question	What a strong answer includes
A Spark job got slower over a month. How do you investigate?	Compare against baseline, check data growth, Spark UI for skew/spill/shuffle, small files plan changes
How would you cut the cost of a nightly pipeline?	Top-cost list first, job vs all-purpose, layout, right-sizing, spot with on-demand driver, benchmark
Partitioning vs Liquid Clustering?	Partition only very large tables with big partitions; Liquid Clustering for flexible, high-cardinality filtering; not combinable on one table
What does AQE do?	Coalesces shuffle partitions, converts joins to broadcast at runtime, splits skewed partitions
How do you handle data skew?	Detect via task time/shuffle-size outliers, AQE skew join, salting, separate hot keys, pre-aggregate
Is Photon always cheaper?	No. Higher DBU rate; benchmark cost per run
Predictive Optimization vs Auto Optimize?	Background table maintenance on UC managed tables vs write-time file sizing
How do you attribute cost to teams?	Tags enforced by policies, `system.billing.usage` plus `list_prices`, cloud cost join, showback
Why doesn't `system.billing.usage` show the whole bill?	Records DBUs only; classic compute VM costs come from the cloud provider
When would you use a materialized view?	Repeated aggregations on incrementally changing sources; verify refresh behavior
How do you prove an optimization worked?	Baseline, single-variable change, multiple runs, median time and cost, regression alert
What is the first thing you check on a cost spike?	Top jobs by cost, new or untagged compute, idle all-purpose clusters, recent layout or code changes



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

Conclusion

Level 6 (Performance & Cost Optimization) is where data engineering becomes economic engineering.

At **Level 5 (Production Readiness & Observability)**, you built systems you could trust and see. At Level 6, you use that visibility to make them lean:

- You read the bill correctly: DBUs plus cloud infrastructure
- You found the 20% of workloads that drive 80% of spend
- You fixed layout before touching configs
- You sized compute with the levers Databricks actually gives you
- You separated automatic maintenance (Predictive Optimization) from write-time tuning (Auto Optimize)
- You attributed cost to teams so improvement had owners
- You proved every change with a baseline and a re-measure

The best performance engineers aren't the ones with the longest list of Spar flags. They measure first, change one thing, and keep only what the numbers support.



Anura

149 foll

Data Er
Exp. Pa
pipeline
perform

What's Next

Fast, cheap and reliable still isn't enough if the wrong person can see the wrong data. The next question is:

Who can access what, how do we protect sensitive data, and how do we prove it to an auditor?

That is **Level 7 (Governance, Security & Lineage).**

Wait for the Next Detailed Blog

Next Monday, 11 AM IST We will go deep into **Level 7 (Governance, Security & Lineage):**

- Unity Catalog as the governance backbone
- Fine-grained access control and privilege design
- Row filters and column masks
- PII classification and masking
- Lineage for debugging and compliance
- Audit logging and access review
- GDPR and regulatory considerations
- Secure data sharing
- Tags, classifications and metadata
- Governance across workspaces at scale



Anura

149 follo

Data Er
Exp. Pa
pipeline
perform

Databricks + | Data Engineering + | Spark + | Big Data +

Python Programming +